

ICML 2024 | 把「到达目标」看成扩散过程的逆过程

McGill 与 Mila 提出 Merlin：一个不需要价值函数的目标条件强化学习方法

在离线目标条件强化学习（offline goal-conditioned RL）中，我们希望智能体只依靠一批预先收集好的数据，就能学会从任意起点到达任意目标。奖励往往是稀疏的：到达目标记 1，否则记 0。这个设定很实用，却也很棘手。

大多数方法的做法是先学一个价值函数（value function），再据此改进策略。但在离线设定下，这一步格外脆弱：策略会尝试数据中从未出现过的动作，价值函数对这些动作给出错误估计，误差在时间上不断累积，最终可能让策略彻底发散。稀疏奖励只会让估计更难。

这篇 ICML 2024 论文（作者来自 McGill University 与 Mila）换了一个角度：如果把目标状态看成我们想要建模的数据分布，那么「到达目标」是不是就等价于扩散模型里的「去噪」？他们提出的方法 Merlin，先构造出一批逐步远离目标的轨迹，再训练一个策略把这个过程反转回来，全程不需要价值函数。

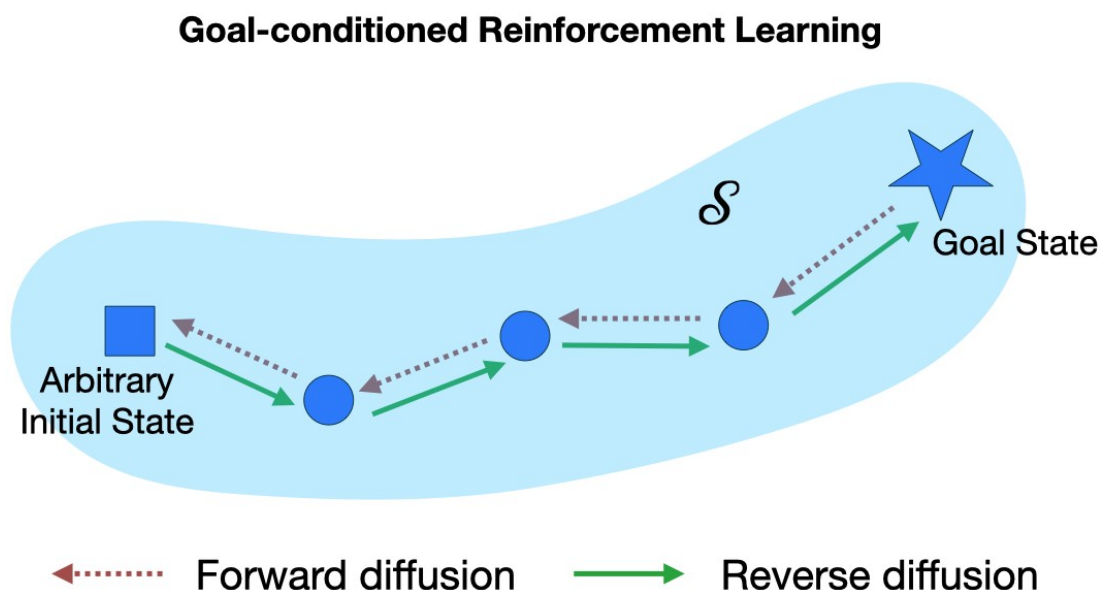


图1 本文方法 Merlin 的核心思想：正如图像扩散把高斯噪声一步步还原成真实样本，Merlin 先构造出从目标「走开」的轨迹（灰色虚线，前向扩散），再学习一个目标条件策略把它们反转（绿色实线，反向扩散），从而能从任意初始状态回到目标。

论文：<https://arxiv.org/abs/2310.02505> | 代码：<https://github.com/vineetjain96/merlin>

为什么价值函数在这里靠不住

在线强化学习可以不断与环境交互、纠正错误的价值估计；离线设定却不行——数据一旦固定，智能体就再也拿不到新的反馈。为了稳住离线学习，已有工作通常给策略加约束，或者采用更保守的价值更新，但这些手段往往以牺牲性能和泛化为代价。问题的根源其实是一个更基本的追问：我们究竟需不需要显式地去估计「每个状态到底有多好」？

扩散模型给出了一个「不需要」的样板。它先用一个前向过程把数据一点点破坏成高斯噪声，再学习一个反向过程，把噪声重新去噪成真实样本——整个过程从头到尾都没有价值函数。Merlin 的出发点正是这个类比：噪声是「走离」数据流形的过程，而在目标条件强化学习里，我们同样可以构造出「走离」目标的轨迹，然后学着把它反转。

Merlin：把目标条件强化学习写成反向扩散

具体怎么做？取数据集里的一条轨迹，从它的最终目标状态开始倒着看。在每一步施加一个前向扩散变换，就会得到一串逐渐远离目标的状态，正如图像扩散里逐步加噪。Merlin 要学的，就是把这个漂移一步步反转回去的目标条件策略。

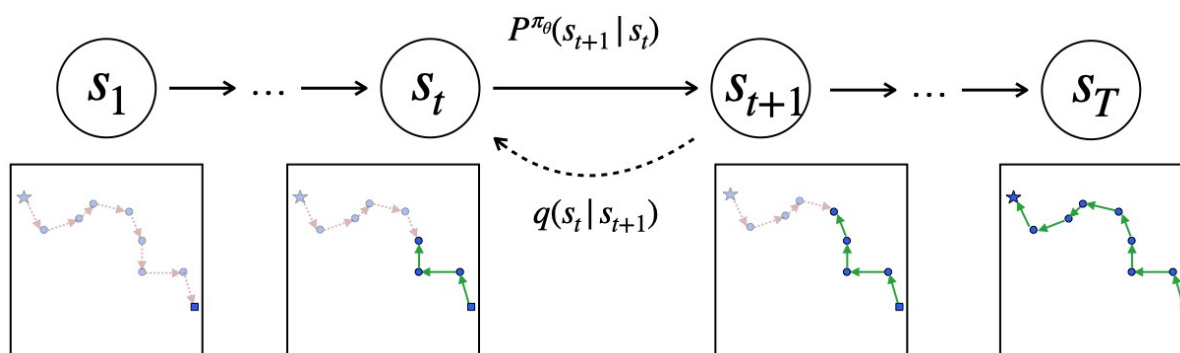


图 2 以 2D 导航为例的前向与反向扩散过程。星号是目标状态；红色虚线箭头表示前向过程的状态转移 $q(s_t | s_{t+1})$ ，绿色箭头表示反向过程 $P^{\pi_\theta}(s_{t+1} | s_t)$ 。策略要学的正是绿色箭头这一方向。

论文的关键理论结果是：在这个反向扩散过程下最大化目标状态的对数似然，等价于在经过事后重标注（hindsight relabeling）的数据上做普通的行为克隆（behavior cloning）。换句话说，价值函数被彻底省掉了，训练目标简化成一条监督学习式的公式。更关键的是，由于扩散直接发生在状态空间里，策略每走一步环境只需要 1 次去噪迭代，而不像典型扩散策略那样每一步都要迭代很多次——这正是 Merlin 又快又稳的来源。

三种「走离目标」的方式

前向轨迹如何构造，直接决定了策略要反转的是什么。Merlin 给出了三种方案，对应三个变体。最基础的 Merlin 用一个固定的启发式规则生成远离目标的轨迹；Merlin-P 把前向过程换成一个可学习的参数化模型（parametric forward model）；Merlin-NP 则采用非参数（non-parametric）的做法——在学到的隐空间里做最近邻检索，把彼此邻近的状态拼接起来。

Merlin-NP 的拼接（stitching）是它更强的关键。因为最近邻是在学到的隐空间里计算的，被连接的状态在语义上彼此靠近，于是反向策略可以把不同轨迹的子段组合起来，拼出一条通向目标的新路径——即使这条完整路径从未在数据中出现过。

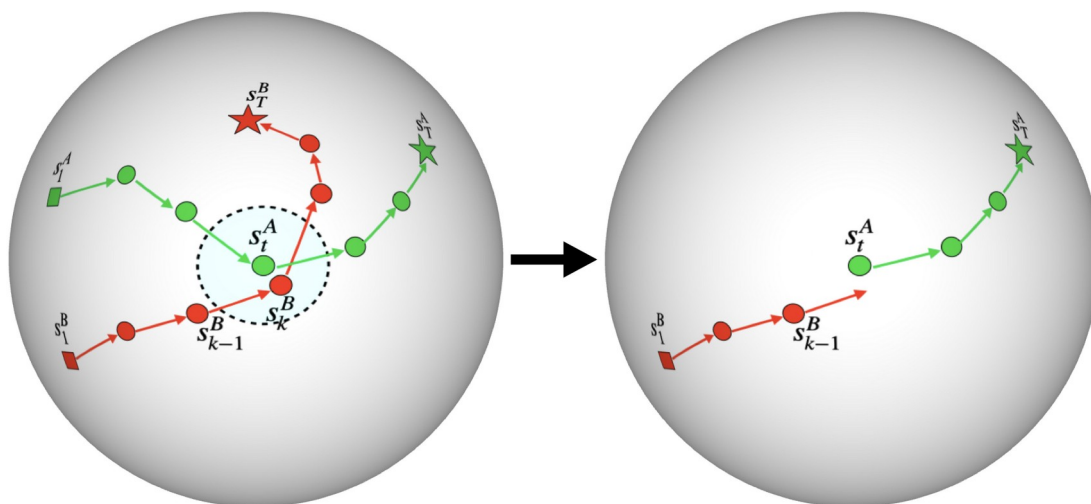


图 3 轨迹拼接：在学到的隐空间中计算最近邻，使被连接的状态彼此邻近（左），从而让反向策略把两条子轨迹拼成一条通向目标的连贯路径（右）。这让 Merlin-NP 能超出单条示范轨迹的覆盖范围。

效果到底如何

作者在 Yang 等人（2021）的离线目标条件基准上评测，一共 10 个控制任务，从简单的点导航、Reacher，到更难的 Fetch 抓取、推动、滑动，以及高维的 HandReach。每个任务都用稀疏二值奖励、最大轨迹长度 $T=50$ ，并在「专家」与「随机」两种数据集上、跨 10 个随机种子取平均。对比对象既包括目标条件强化学习基线（GCSL、WGCSL、AM、GoFAR），也包括扩散类方法（Decision Diffuser、g-DQL、BESO）。

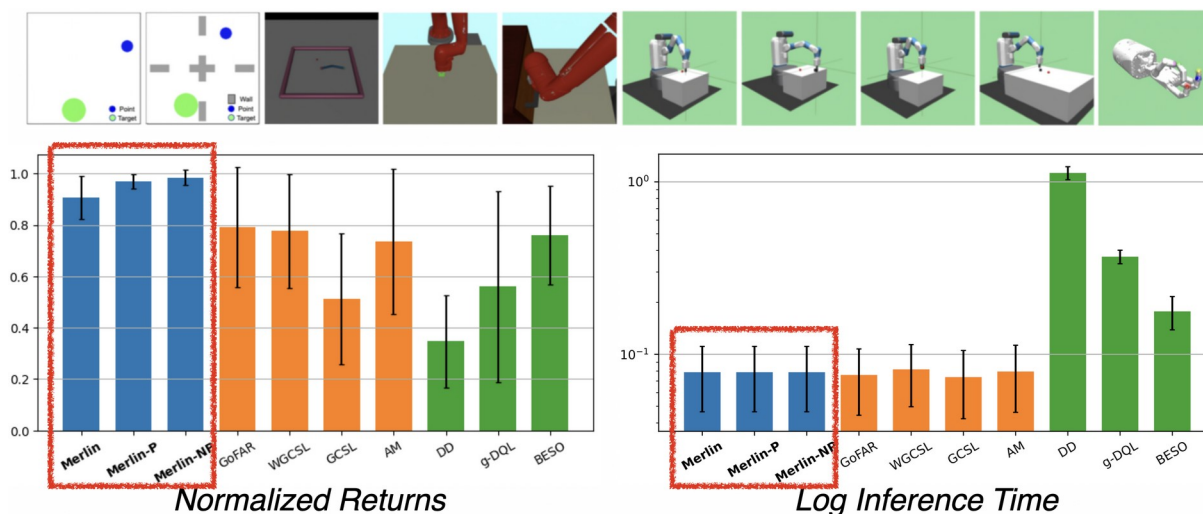


图 4 基准任务（上）、归一化回报（左）与对数推理时间（右）。Merlin、Merlin-P、Merlin-NP（蓝色，红框标出）在回报上追平或超过所有基线，同时推理速度比扩散类方法（DD、BESO）快约一个数量级（10×）。

结论相当一致：最基础的 Merlin 就已经在多数任务上胜过基线，Merlin-P 与 Merlin-NP 进一步把回报推高。以在全部 10 个方法、10 个任务上的平均排名来衡量，Merlin-NP 综合最优——在状态观测下平均排名约 1.7，在像素观测下约 1.25。而由于每步只做一次去噪，它的训练和推理都比 Decision Diffuser、BESO 这类扩散方法快约 10×；观测维度越高（例如像素输入），这个速度差距还会进一步拉大。

表 1 三个 Merlin 变体及其前向轨迹的构造方式。

变体	前向轨迹构造方式
Merlin	固定的启发式规则
Merlin-P	可学习的参数化前向模型
Merlin-NP	隐空间最近邻拼接（非参数）

该往前看多远

Merlin 有两个主要的可调超参数：事后重标注的比例，以及评测时的时间视野 h （策略每一步瞄准多远的目标）。作者系统地扫描了 h 的取值，并把结果画成热力图。

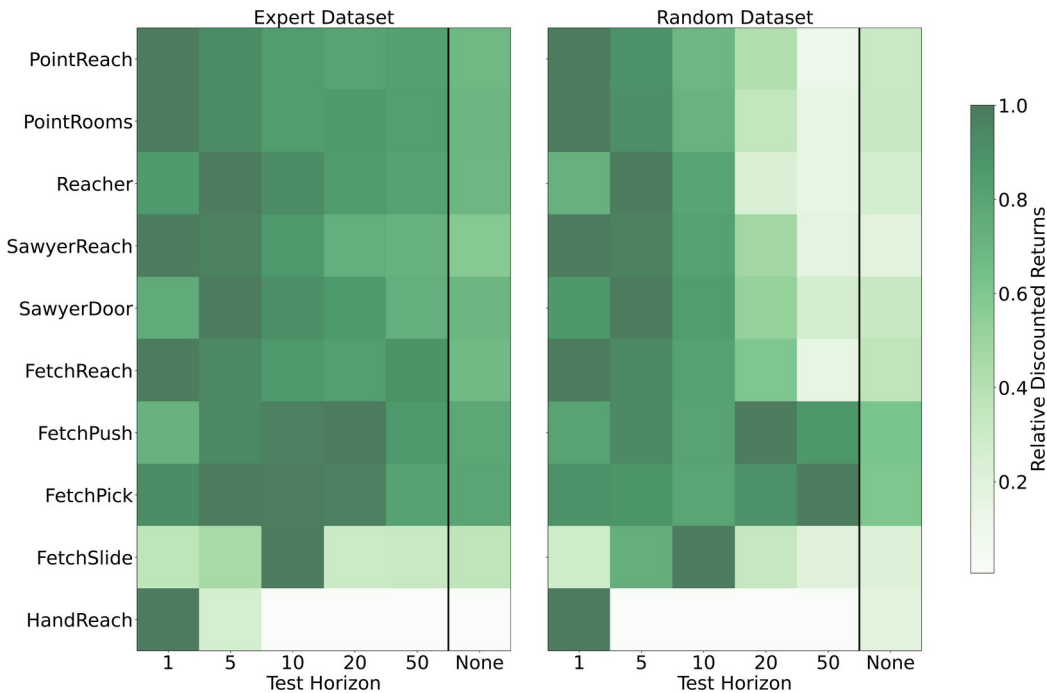


图 5 不同评测视野 h 下各任务的相对折扣回报（左：专家数据集；右：随机数据集）。总体上「条件于某个视野」都优于「不条件」（None 列），且最优 h 随任务难度变化：简单任务偏好短视野，HandReach 尤其明显，只有在 $h=1$ 时才表现良好。

从热力图可以读出两点。其一，条件于一个时间视野，几乎总是优于完全不条件（最右侧 None 列颜色更浅）。其二，最优视野取决于任务：像 PointReach 这样简单的任务，在短视野（ $h=1$ 或 $h=5$ ）下就很好，而更难的任务需要更长的视野；扫描范围大致覆盖 $\{1, 5, 10, 20,$

50}。HandReach 是最敏感的一个——只有在 $h=1$ 时才有效。这说明 Merlin 对视野的选择整体是稳健的，但个别高维任务仍需针对性调参。

小结与边界

Merlin 最值得记住的一点，是它把离线目标条件强化学习重写成了一个扩散过程的逆过程，而这个重写让问题变得异常简单：构造出远离目标的轨迹、再学着反转它们，目标到达就退化成了普通的行为克隆——没有价值函数要估计，也没有随之而来的不稳定要对抗。它在 10 个任务上追平或超过当前最好的方法，却比其它扩散类方法快约一个数量级。

当然，这套框架也有它的前提。它面向的是离线、稀疏奖励、目标条件的设定；前向轨迹的构造方式（固定、参数化，或非参数拼接）需要按任务选择，个别高维任务（如 HandReach）对视野等超参数仍然敏感。但把扩散直接放在状态空间、每步只做一次去噪这一点，让「用扩散来做强化学习」第一次显得既简单又可扩展——这也许比某个具体分数更值得关注。